A INTERNSHIP REPORT ON

NEW SPARTANS

A FITNESS TRACKING MOBILE APPLICATION

Submitted by

FARISH SHARIF A 510623205017

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

C. ABDUL HAKEEM COLLEGE OF ENGINEERING AND TECHNOLOGY, MELVISHARAM, RANIPET - 632509

ANNA UNIVERSITY::CHENNAI - 600 025

BATCH 2023-2027

INTERNSHIP OFFER LETTER

BRIDGE TO CORPORATE

21/1, 1st Floor, 3rd Main, 8th Cross, Madiwala, Bangalore - 560068

OFFER LETTER FOR INTERNSHIP PROGRAM

To:
FARISH SHARIF A
Phone: +91 94872 33290
Email: info@farishsharif.in

Date: 08-June-2026
Offer Letter No: BTC/INT/2026/354

Dear FARISH,

We are pleased to inform you that you have been selected for the Internship Program at Bridge To Corporate - Training & Consultancy based on your application and interaction with our selection team.
You are hereby offered the position of Intern - Mobile App devlopment under our internshipprogram as per the following terms and conditions:

1. Internship Details
Internship Role: Mobile App devlopment
InternDepartment: IT
Internship Duration: 15 June 2026 To 18 July 2026

2. Scope of Work & Training
During the internship, you will be trained and assigned practical tasks related to:

- Work on live Mobile App devlopment projects and gain practical
- experience. Learn and use real-time Mobile App devlopment tools and
- technologies.Complete coding assignments, assessments, and mini
- projects.Prepare project documentation and progress reports.

The internship is designed for learning, skill development, and industry exposure in Mobile App devlopment.

3. Stipend
This is a Non-Paid, Skill-Based Internship offered purely for academic and practical learning purposes. This internship does not constitute employment with the company.

4. Confidentiality Clause
All data, documents, training materials and project information shared during the internship are strictly confidential and must not be disclosed to any third party during or after completion of the internship.

+91-81055 77689 info@bridgetocorp.com www.bridgetocorp.com

5. Performance & Certification
On successful completion of the internship:
You will be issued an Internship Completion Certificate.

If applicable, Project Certificate, LOR (Letter of Recommendation) and Experience Letter may be issued based on performance.
Final certification will be subject to:

- Minimum 80% attendance
- Successful project submission
- Satisfactory performance review

6. Rules & Code of Conduct
You are required to:

- Maintain professional behavior, discipline, and punctuality
- Follow all company policies, data security rules, and confidentiality guidelines
- Not share internal materials without written permission
- Complete tasks and reporting within assigned deadlines

Any misconduct may lead to immediate termination of the internship without certification.

7. Termination Clause
Either party may terminate the internship by giving 7 days written notice.
In case of serious misconduct or violation of rules, the internship may be terminated immediately without notice or certification.

8. Acceptance of Offer
You are requested to confirm your acceptance of this offer by signing and returning a copy of this letter on or before 12-June-2026.

For Bridge To Corporate – Training & Consultancy
Authorized Signatory:

Name: Imran Khan
Designation: Founder / Proprietor
Signature: imran khan
Seal:

I, FARISH SHARIF A, hereby accept the above internship offer and agree to abide by all the terms and conditions of the internship program.

Student Signature: _____
Date:
Place:

INTERNSHIP CERTIFICATE



The above certificate was awarded by Bridge To Corporate - Training & Consultancy on successful completion of the Mobile Application Development internship from 15 June 2026 to 18 July 2026.

ACKNOWLEDGEMENT

I express my sincere gratitude to the Management and Correspondent of C. Abdul Hakeem College of Engineering and Technology for providing the opportunity to undertake this internship and gain practical exposure to mobile application development. The academic environment and encouragement I received throughout my course gave me the confidence to take up and complete this work.

I sincerely thank our Principal **Dr. M. Sasi Kumar, Ph.D.,** and Vice Principal **Dr. Mohammed Muzaffer Hussain, Ph.D.,** for their encouragement and for providing the facilities and academic support needed to complete this internship report.

I extend my heartfelt thanks to our Head of the Department, **Dr. S. Umamaheswari, M.Tech., Ph.D.,** for her constant guidance and support. I also thank the **IT Department Staffs** for their valuable advice and encouragement during the internship and the preparation of this report.

I am grateful to **Mr. Imran Khan, Founder of Bridge To Corporate - Training & Consultancy,** for providing the internship opportunity. I thank my project mentor, **Mr. Kashish Gowda,** and the entire team for their guidance, timely feedback and support throughout the internship, which helped me understand real-world mobile development practices.

The project, **New Spartans - A Fitness Tracking Mobile Application,** provided an opportunity to relate classroom concepts to a working mobile product. Preparing this report helped me organise the requirements, interface design, development environment and visual outputs in a structured manner.

Finally, I thank my parents and everyone who supported and encouraged me during the internship and the preparation of this report.

ABSTRACT

This report presents New Spartans, a fitness tracking mobile application developed during a mobile application development internship at Bridge To Corporate - Training & Consultancy from 15 June 2026 to 18 July 2026. The project is built with the Flutter framework and the Dart programming language and adopts a modern, dark, mobile-first interface. It supports a workouts dashboard, detailed workout screens with muscle-group progress, live workout tracking with a timer and exercise completion, body-weight logging, an activity calendar, lifted-volume statistics, personal records, a rule-based AI coach chat and an editable user profile.

The application uses the Provider package (ChangeNotifier) to centralise state, so that every screen reflects the same workout, statistics and history information. In-memory model data supplies the catalogue of workouts, exercises, muscle groups and statistics, while calculation functions derive volume lifted, completion counts and streaks. The report describes the company, the project objectives, the system design, the technologies learnt, the supplied screenshots, the reported manual testing and representative source-code excerpts.

The work provided practical experience in widget composition, typed data modelling, state management, responsive interface design, debugging and technical documentation. The application is a client-side prototype: a live backend, authenticated accounts and persistent cloud storage are identified as future extensions.

TABLE OF CONTENTS

CHAPTER NO TITLE PAGE NO

ABSTRACT....vi

LIST OF FIGURES....ix

LIST OF TABLES....x

1 INTRODUCTION ABOUT THE COMPANY....1

1.1 COMPANY PROFILE....1

1.2 INTERNSHIP PARTICULARS....1

1.3 TRAINING AND PROFESSIONAL LEARNING....1

1.4 OBJECTIVES OF THE INTERNSHIP....2

2 INTRODUCTION....3

2.1 PROJECT OVERVIEW....3

2.2 PROBLEM STATEMENT....3

2.3 OBJECTIVES....3

2.4 SCOPE OF THE REPORT....3

2.5 EXISTING AND PROPOSED SYSTEM....4

3 PROJECT DESCRIPTION....5

3.1 SYSTEM ANALYSIS....5

3.2 ARCHITECTURE DIAGRAM....7

3.3 DATA DESIGN....8

3.4 STATE MANAGEMENT AND NAVIGATION....9

4 TECHNOLOGY LEARNT....10

4.1 MOBILE TECHNOLOGIES....10

4.2 DEVELOPMENT AND TOOLING....10

4.3 HARDWARE AND SOFTWARE REQUIREMENTS....10

4.4 LEARNING OUTCOMES....11

5 RESULTS AND OUTPUTS....12

5.1 WORKOUTS DASHBOARD AND STATS....12

5.2 WORKOUT DETAIL AND MUSCLE GROUPS....12

5.3 LIVE WORKOUT TRACKING....14

5.4 PROGRESS, CALENDAR, COACH AND PROFILE....14

5.5 WORKOUT LOGIC AND CALCULATIONS....15

5.6 TESTING AND DEVELOPMENT WORKFLOW.....17
6 CONCLUSION.....18
FUTURE WORK.....18
REFERENCES.....19
APPENDIX - SOURCE CODE.....20

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.1	Workout Flow of New Spartans	7
5.1	Workouts Dashboard Screen	12
5.2	Workout Detail Screen	12
5.3	Live Workout Tracking Screen	14
5.4	Profile Screen	14

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
1.1	Internship Particulars	1
3.1	Functional Modules and Responsibilities	5
3.2	Functional and Non-Functional Requirements	5
3.3	Workout, Exercise and Stats Data	8
4.1	Development Commands	10
4.2	Hardware and Software Requirements	11
5.1	Workout Calculations from Supplied Source	15
5.2	Reported Manual Test Cases	17

CHAPTER 1
INTRODUCTION ABOUT THE COMPANY

1.1 COMPANY PROFILE

Bridge To Corporate - Training & Consultancy is based at 21/1, 1st Floor, 3rd Main, 8th Cross, Madiwala, Bangalore - 560068. The organisation provides practical training and skill-based internship programmes for engineering and technology students. It focuses on bridging the gap between academic study and industry expectations by offering guided, project-based learning.

The company conducts internships in areas such as web development, mobile application development, data handling and user-interface design. Its mobile application development internship connects academic learning with real coding assignments, mentor-guided project work and structured documentation, so that interns gain exposure to tools, workflows and delivery practices used in professional software teams.

1.2 INTERNSHIP PARTICULARS

Table 1.1 Internship particulars

PARTICULAR	DETAIL
Student / register number	Farish Sharif A / 510623205017
Course	B.Tech Information Technology, Batch 2023-2027
Organisation	Bridge To Corporate - Training & Consultancy
Role / department	Mobile Application Development Intern / IT
Duration	15 June 2026 to 18 July 2026
Offer letter reference	BTC/INT/2026/354, dated 08 June 2026
Project mentor	Mr. Kashish Gowda
Project title	New Spartans - A Fitness Tracking Mobile Application
Contact email	info@farishsharif.in

1.3 TRAINING AND PROFESSIONAL LEARNING

The internship involved mobile application development tasks, use of modern tools, coding assignments, mini projects and regular progress reporting. In the early phase, the focus was on understanding the Flutter framework, the Dart language and the widget model. In later phases, the work moved to building complete screens, managing shared application state and refining the user interface.

Under the guidance of the project mentor and the team, the New Spartans project provided practical experience in mobile UI architecture, state management, responsive interface design and technical documentation. Weekly feedback sessions helped identify issues early and encouraged clean, maintainable code organisation.

1.4 OBJECTIVES OF THE INTERNSHIP

1. To gain hands-on experience in mobile application development using Flutter and Dart.
2. To learn state management and clean code organisation in a real project.
3. To practise building responsive, user-friendly mobile interfaces.
4. To understand the software development workflow, from requirements to testing.
5. To develop professional skills such as documentation, reporting and time management.

CHAPTER 2
INTRODUCTION

2.1 PROJECT OVERVIEW

New Spartans is a fitness tracking mobile application that combines workout planning, live session tracking, progress statistics and an AI coach in a single app. Flutter widgets and shared application state connect the screens into a smooth mobile experience with a bottom navigation bar for quick movement between the main sections.

The app is designed for people who train regularly and want a simple, distraction-free way to plan their sessions, record their effort and watch their progress over time. The dark theme keeps the focus on the content and is comfortable to use in a gym environment.

2.2 PROBLEM STATEMENT

Many people who exercise rely on paper notes or memory to track their workouts, which makes it hard to follow a plan, measure progress or stay motivated. A useful fitness app should organise workout information clearly, track sets, reps and weights consistently, and keep session data available across all screens.

New Spartans addresses these needs as a client-side mobile prototype. It provides a structured way to view workouts, run a live session, record completed exercises and review statistics, so that the user always has an accurate picture of their training.

2.3 OBJECTIVES

1. Design a fitness app with consistent navigation and a clean dark theme.
2. Support workout browsing, workout detail and muscle-group progress tracking.
3. Track live workouts with a timer, exercise completion and volume lifted.
4. Provide body-weight logging, an activity calendar, statistics and an AI coach.
5. Develop responsive layouts and document the project and its outputs.

2.4 SCOPE OF THE REPORT

The project uses static, in-memory model data held in application state. It does not connect to a live database or a cloud account service, and the AI coach uses rule-based replies rather than an external language model. Screenshots show the mobile outputs, while the supplied code

explains application state and calculation logic. Backend integration, authentication and persistent storage remain future work and are discussed in the conclusion.

2.5 EXISTING AND PROPOSED SYSTEM

Existing habits for tracking workouts often rely on notebooks, generic note apps or spreadsheets. These approaches do not calculate progress automatically, do not provide a live session view and do not present statistics in a convenient form on a mobile device.

The proposed system, New Spartans, brings these functions together in one mobile app. It stores workouts and exercises as structured data, calculates volume lifted and completion automatically, marks an activity calendar when sessions are finished and presents progress through clear visual cards. This reduces manual effort and gives the user immediate feedback on their training.

CHAPTER 3
PROJECT DESCRIPTION

3.1 SYSTEM ANALYSIS

The application uses a central WorkoutProvider, implemented as a ChangeNotifier, to coordinate workouts, muscle groups, user statistics, the live session, completed-workout history, the user profile and chat messages. The MainNavigationScreen hosts the bottom navigation and switches between the Workouts, Calendar, Stats and Chat screens, while the workout detail and live-tracking screens are pushed on demand. This separation keeps each screen focused while the provider holds the shared state.

Table 3.1 Functional modules and responsibilities

MODULE	RESPONSIBILITY
WorkoutProvider	Shared app state: workouts, muscle groups, statistics, live session, history and profile.
Screens	Workouts, Workout Detail, Workout In Progress, Calendar, Stats, Chat, Profile and Add/Edit.
Widgets	WorkoutCard, StatsCard, CalendarCard, MuscleGroupCard, ExerciseListItem and others.
Models	Workout, Exercise, MuscleGroup, UserStats, CompletedWorkout, PersonalRecord and UserProfile.
Theme	Centralised dark theme, colours and text styles defined in AppTheme.

3.1.1 Interface Requirements

The app needs readable workout cards, clear muscle-group progress indicators, an easy live-tracking flow and visible feedback after each action. A common theme and reusable widgets maintain visual consistency, and layouts must adapt to different device sizes without clipping content.

3.1.2 Functional and Non-Functional Requirements

Table 3.2 Functional and non-functional requirements

FUNCTIONAL	NON-FUNCTIONAL
View, add and open workouts.	Responsive layout across device sizes.
Start, track and finish a live session.	Smooth animations and fast screen switching.
Record body weight and view history.	Consistent dark theme and readable text.
Show statistics, calendar and records.	Reliable state updates via notifyListeners.
Chat with the AI coach and edit profile.	Clean, maintainable and testable code.

3.1.3 Quality Requirements

Exercise completion and finishing a workout must produce consistent statistics. Finishing a session must save it to history and mark the activity calendar. The interface must refresh reliably whenever the shared state changes. Production use would additionally require secure accounts, a backend and validated data storage.

3.2 ARCHITECTURE DIAGRAM

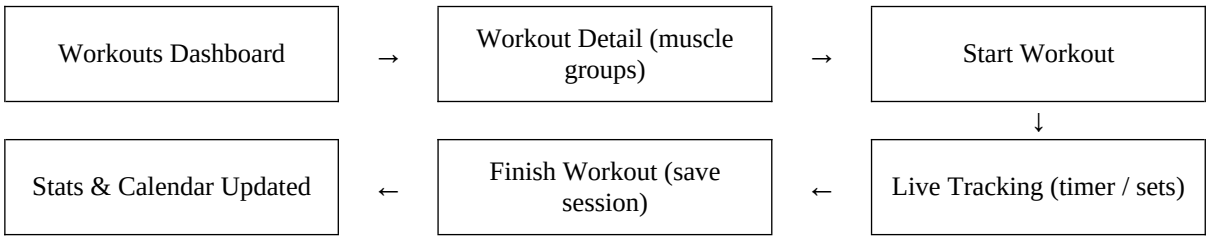


Figure 3.1 Workout flow of New Spartans

The entry file main.dart runs NewSpartansApp, which wraps MainNavigationScreen in a ChangeNotifierProvider of WorkoutProvider. The user browses workouts, opens a workout, starts a live session, completes exercises and finishes. The shared state then updates statistics, history and the activity calendar, and every dependent screen rebuilds to show the new values.

3.3 DATA DESIGN

The data layer is made up of plain Dart model classes. They describe workouts and their exercises, the tracked muscle groups, user statistics and the records kept for progress. The models are immutable where possible, and copyWith methods are used to create updated copies safely.

Table 3.3 Workout, exercise and stats data

ENTITY	MAIN INFORMATION
Workout	id, name, day, duration (minutes), type (Push/Pull/Cardio) and a list of exercises.
Exercise	id, name, sets, reps, weight (kg), notes, isCompleted and type (weight, cardio, bodyweight, time).
Muscle group	name, icon, status, current / target / total reps and progress (0.0 to 1.0).
UserStats	body weight, last update, volume lifted, period and the workout calendar map.
Records	completed workouts, body-weight history and personal records.
UserProfile	name, bio, goal, target weight, age and height.

The sample data contains three workouts (Push, Pull and Cardio) made up of fifteen exercises, six tracked muscle groups and generated statistics such as body-weight history and personal records. The records are held in application state; there is no database server. Workout history and statistics update in memory as the user completes sessions.

3.4 STATE MANAGEMENT AND NAVIGATION

State management is handled by the Provider package. A single WorkoutProvider extends ChangeNotifier and exposes the application data through getters, together with methods that change the data. Screens read values with context.watch or a Consumer and trigger changes by calling provider methods. After each change the provider calls notifyListeners, and only the widgets that depend on the changed data rebuild.

Navigation uses a bottom navigation bar hosted by MainNavigationScreen. It switches between the Workouts, Calendar, Stats and Chat screens, while detailed screens such as the workout detail, live tracking, add workout, profile and edit profile are pushed onto the navigation stack. This keeps the main sections one tap away and the detailed flows contained within their own routes.

CHAPTER 4
TECHNOLOGY LEARNT

4.1 MOBILE TECHNOLOGIES

Flutter is an open-source UI toolkit from Google that builds natively compiled applications for mobile, web and desktop from a single codebase. It uses a widget tree, where everything on screen is a widget, and rebuilds efficiently when state changes. During the internship, I learnt how to compose layouts using widgets such as Scaffold, Column, Row, ListView, Card and custom widgets.

Dart is the programming language used by Flutter. It is strongly typed and supports classes, generics, async programming and null safety. The project uses Dart classes to define the Workout, Exercise, MuscleGroup, UserStats and related models, and uses enums such as ExerciseType to represent fixed sets of values.

Material Design and a custom AppTheme supply the dark theme, colours, spacing and text styles. The Provider package (ChangeNotifier) exposes shared application state through a WorkoutProvider. The flutter_animate package adds smooth transitions, the intl package handles date and number formatting, and flutter_launcher_icons generates the launcher icon for the installed app.

4.2 DEVELOPMENT AND TOOLING

The project was developed using the Flutter SDK with Android Studio and VS Code. Common commands were used throughout the development cycle to install dependencies, run the app, run tests and generate assets.

Table 4.1 Development commands

COMMAND / TOOL	PURPOSE
flutter pub get	Install the project dependencies.
flutter run	Build and launch the app on an emulator or device.
flutter test	Run the unit and widget tests.
flutter pub run flutter_launcher_icons:main	Generate the app launcher icons.
flutter build apk	Build a release Android package.
Android Studio / VS Code	Edit code, inspect widgets and debug behaviour.

4.3 HARDWARE AND SOFTWARE REQUIREMENTS

Table 4.2 Hardware and software requirements

REQUIREMENT	DETAIL
Processor	Dual-core processor or better.
Memory	At least 8 GB RAM recommended for the emulator.
Operating system	Windows, macOS or Linux development machine.
Flutter SDK	Flutter 3.8.1 or later with the Dart SDK.
Editor	Android Studio or VS Code with the Flutter plugin.
Target device	Android or iOS device or emulator.

4.4 LEARNING OUTCOMES

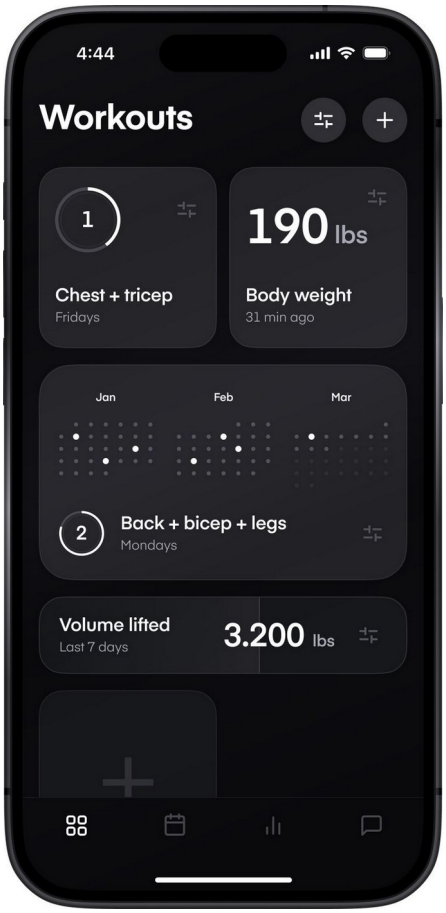
The project developed skills in widget composition, typed data modelling, state management with Provider, calculation functions, responsive styling and incremental testing. It also strengthened my understanding of project structure, separating code into screens, widgets, providers, models and theme. Working to a schedule with mentor feedback improved my debugging, documentation and time-management abilities.

CHAPTER 5
RESULTS AND OUTPUTS

5.1 WORKOUTS DASHBOARD AND STATS

The Workouts screen is the home of the app. It shows workout cards with their scheduled days, a body-weight tile, an activity calendar across recent months and a volume-lifted summary, with a dedicated card to add a new workout. The cards use the shared dark theme and give the user an at-a-glance view of their training.

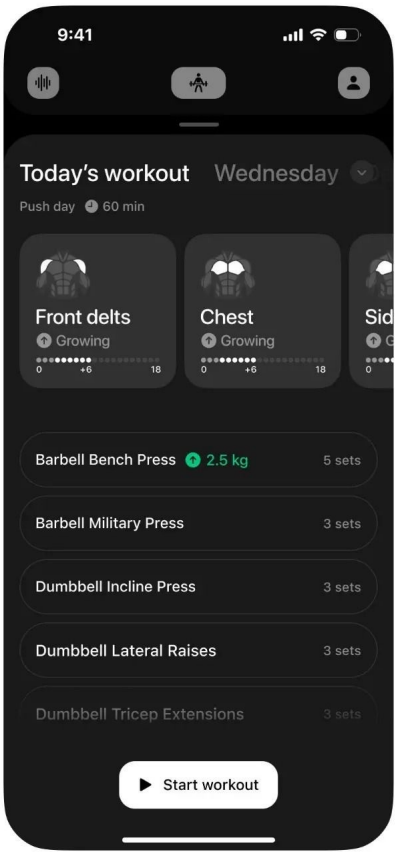
Figure 5.1 Workouts dashboard screen



5.2 WORKOUT DETAIL AND MUSCLE GROUPS

The workout detail screen shows the day, type and duration, a horizontal row of muscle-group cards with progress indicators, and the exercise list with sets, reps and weight changes. A Start Workout button begins a live session. The progress indicators show how each muscle group is developing across recent sessions.

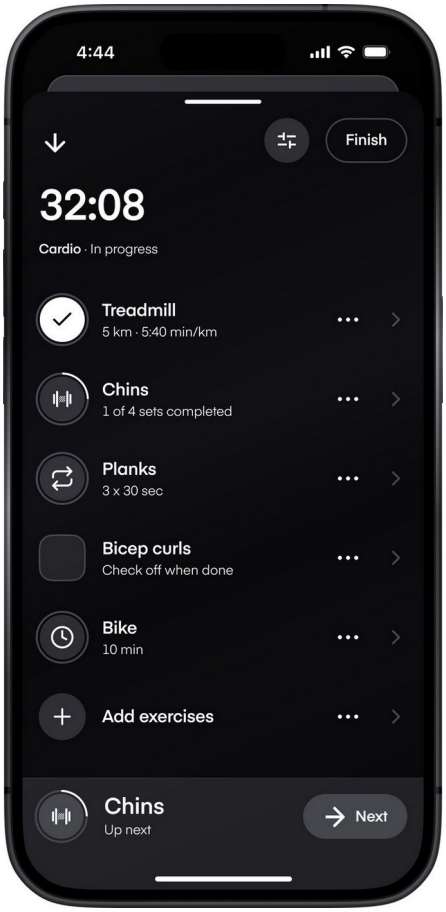
Figure 5.2 Workout detail screen


9:41
Today's workout
Wednesday
Push day
60 min
Front delts
Growing
0
+6
18
Chest
Growing
0
+6
18
Barbell Bench Press
2.5 kg
5 sets
Barbell Military Press
3 sets
Dumbbell Incline Press
3 sets
Dumbbell Lateral Raises
3 sets
Dumbbell Tricep Extensions
3 sets
Start workout

5.3 LIVE WORKOUT TRACKING

During a session, a running timer and the current workout type are shown at the top. Each exercise can be marked complete, sets are tracked, and the next exercise is previewed at the bottom. A Finish action ends the session, saves it to history and marks the activity calendar. The checklist makes it easy to follow the plan without losing the place.

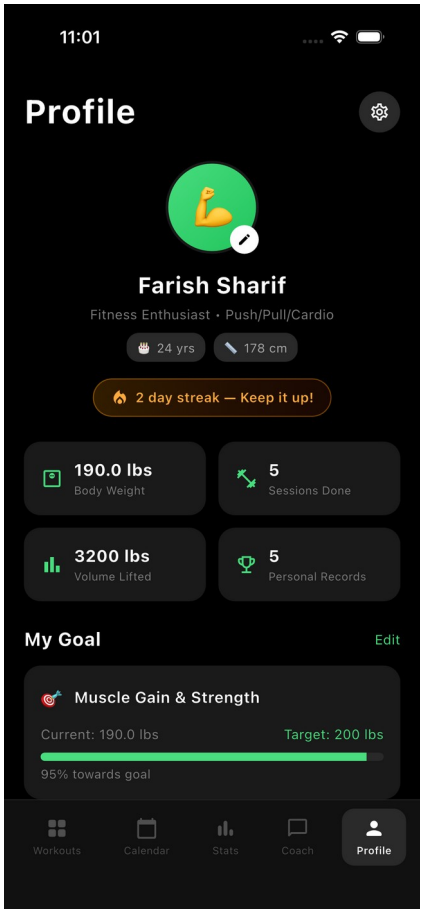
Figure 5.3 Live workout tracking screen



5.4 PROGRESS, CALENDAR, COACH AND PROFILE

Beyond the dashboard, the Stats screen summarises body-weight history, lifted volume and personal records using cards and simple charts. The Calendar screen visualises workout frequency over recent months. The Chat screen provides a rule-based AI coach that answers questions on training, nutrition and recovery with helpful, context-aware replies. The Profile screen displays the user profile and allows the goal, bio and body details to be edited and saved.

Figure 5.4 Profile screen



5.5 WORKOUT LOGIC AND CALCULATIONS

The supplied WorkoutProvider code defines the tracking and statistics logic. These rules describe the prototype implementation and operate on in-memory data; they do not establish a live account or server record.

Table 5.1 Workout calculations from supplied source

ITEM	IMPLEMENTED RULE
Volume lifted	Sum of weight x sets x reps for weight-type exercises; cardio and bodyweight contribute zero.
Workout streak	Counts consecutive recent days (up to 30) that have a completed workout in the calendar.
Completion	On finish, exercises marked done are counted and shown as completed out of total.
Body weight	logBodyWeight adds a dated entry and updates the current body-weight statistic.
Activity calendar	Finishing a workout marks the current date, and the dashboard dots reflect marked days.
Session volume	On finish, the session volume is added to the running volume-lifted total.

5.6 TESTING AND DEVELOPMENT WORKFLOW

The source report records manual validation on an Android emulator and a physical device at different screen sizes. The following outcomes reproduce those reported results; they are not new test executions performed during report formatting.

Table 5.2 Reported manual test cases

TEST CASE	EXPECTED BEHAVIOUR	REPORTED
Add Workout	A new workout appears on the dashboard list.	Pass
Start Workout	The live tracking screen opens with the timer at zero.	Pass
Complete Exercise	Tapping an exercise marks it done and advances the next item.	Pass
Finish Workout	The session is saved and today is marked on the calendar.	Pass
Log Body Weight	A new weight updates the dashboard and weight history.	Pass
Edit Profile	Updated profile details persist on the profile screen.	Pass
AI Coach Chat	The coach returns a relevant reply for a workout query.	Pass
Responsive Layout	Layout adapts across device sizes.	Pass

Development Observations

The source report describes fixes for statistics that did not refresh after finishing a session, the exercise index not advancing on completion and inconsistent colours across screens. The recorded solutions were calling notifyListeners after state updates, deriving the next exercise from the completed count and centralising colours and text styles in AppTheme.

Automated unit and widget tests, a live backend, authentication and persistent storage are listed as future development.

CHAPTER 6
CONCLUSION

New Spartans demonstrates a responsive fitness tracking mobile application built with Flutter and Dart. Its widget structure and shared state connect workout planning, muscle-group progress, live tracking, statistics, an activity calendar, an AI coach and an editable profile into one coherent product.

The project provided practical experience in typed data modelling, reusable widgets and state management with Provider. Centralised calculations coordinate volume lifted, completion, body weight and the activity calendar, while in-memory state supports the prototype without introducing a server database. Organising the code into screens, widgets, providers, models and theme clarified the responsibility of each part of the application.

The supplied screenshots document the workouts dashboard, workout detail and live tracking screens. The source report records manual tests for workout operations, live tracking, statistics, chat and responsive behaviour. The appendix retains representative application-shell, theme, model, provider, screen and widget code. The internship strengthened my debugging, documentation and time-management skills through incremental screen development and mentor guidance.

FUTURE WORK

Future work can add a backend, persistent storage and user accounts, cloud synchronisation of workouts and a smarter AI coach backed by a language model. Automated unit and widget testing would improve confidence in state transitions and calculations. Production deployment should also address accessibility, security, performance and app-store readiness. The current work remains a client-side prototype rather than a live, account-based service, but it provides a solid foundation for these extensions.

REFERENCES

[1] Flutter Documentation. Google. https://docs.flutter.dev

[2] Dart Documentation. Google. https://dart.dev/guides

[3] Provider Package Documentation. https://pub.dev/packages/provider

[4] Material Design Guidelines. Google. https://m3.material.io

[5] flutter_animate Package. https://pub.dev/packages/flutter_animate

[6] intl Package. https://pub.dev/packages/intl

[7] flutter_launcher_icons Package. https://pub.dev/packages/flutter_launcher_icons

[8] New Spartans - Project Source Code. https://github.com/farish-sharif/new_spartans

[9] Bridge To Corporate - Training & Consultancy. Internship offer BTC/INT/2026/354, dated 08 June 2026; internship certificate, 18 July 2026.

[10] Farish Sharif A. Supplied New Spartans report, project screenshots and source-code excerpts, 2026.

APPENDIX

SOURCE CODE

This appendix contains representative source code from the New Spartans project, covering the application shell, theme, data models, state management, a screen and a widget.

A.1 Application shell (lib/main.dart)

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'theme/app_theme.dart';
import 'providers/workout_provider.dart';
import 'screens/main_navigation_screen.dart';

void main() {
  runApp(const NewSpartansApp());
}

class NewSpartansApp extends StatelessWidget {
  const NewSpartansApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => WorkoutProvider(),
      child: MaterialApp(
        title: 'New Spartans',
        theme: AppTheme.darkTheme,
        home: const MainNavigationScreen(),
        debugShowCheckedModeBanner: false,
      ),
    );
  }
}
```

A.2 Theme (lib/theme/app_theme.dart)

```
import 'package:flutter/material.dart';

class AppTheme {
  static const Color primaryColor = Color(0xFF000000);
  static const Color cardColor = Color(0xFF1A1A1A);
  static const Color secondaryColor = Color(0xFF2A2A2A);
  static const Color accentColor = Color(0xFF00FF00);
  static const Color textPrimary = Color(0xFFFFFFFF);
  static const Color textSecondary = Color(0xFFB0B0B0);
  static const Color textTertiary = Color(0xFF808080);

  static ThemeData get darkTheme {
    return ThemeData(
      brightness: Brightness.dark,
      scaffoldBackgroundColor: primaryColor,
      primaryColor: primaryColor,
      colorScheme: const ColorScheme.dark(
        primary: primaryColor,
        secondary: accentColor,
        surface: cardColor,
        background: primaryColor,
        onPrimary: textPrimary,
        onSecondary: primaryColor,
        onSurface: textPrimary,
        onBackground: textPrimary,
      ),
      appBarTheme: const AppBarTheme(
        backgroundColor: primaryColor,
        foregroundColor: textPrimary,
        elevation: 0,
        centerTitle: false,
        titleTextStyle: TextStyle(
          color: textPrimary,
          fontSize: 24,
          fontWeight: FontWeight.bold,
        ),
      ),
      textTheme: const TextTheme(
        headlineLarge: TextStyle(
          color: textPrimary,
          fontSize: 32,
          fontWeight: FontWeight.bold,
        ),
        headlineMedium: TextStyle(
          color: textPrimary,
          fontSize: 24,
          fontWeight: FontWeight.bold,
        ),
        headlineSmall: TextStyle(
          color: textPrimary,
          fontSize: 20,
          fontWeight: FontWeight.w600,
        ),
        bodyLarge: TextStyle(
          color: textPrimary,
          fontSize: 16,
          fontWeight: FontWeight.normal,
        ),
        bodyMedium: TextStyle(
          color: textSecondary,
          fontSize: 14,
          fontWeight: FontWeight.normal,
        ),
        bodySmall: TextStyle(
          color: textTertiary,
          fontSize: 12,
          fontWeight: FontWeight.normal,
        ),
      ),
      cardTheme: CardThemeData(
        color: cardColor,
        elevation: 0,
```

```
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
      ),
    ),
    elevatedButtonTheme: ElevatedButtonThemeData(
      style: ElevatedButton.styleFrom(
        backgroundColor: textPrimary,
        foregroundColor: primaryColor,
        elevation: 0,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(12),
        ),
        padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      ),
    ),
    bottomNavigationBarTheme: const BottomNavigationBarThemeData(
      backgroundColor: cardColor,
      selectedItemColor: textPrimary,
      unselectedItemColor: textTertiary,
      type: BottomNavigationBarType.fixed,
      elevation: 0,
    ),
  );
 }
}
```

A.3 Data models (lib/models/workout.dart)

```
class Workout {
  final String id;
  final String name;
  final String day;
  final int duration; // in minutes
  final List<Exercise> exercises;
  final String type; // e.g., "Push day", "Cardio"

  Workout({
    required this.id,
    required this.name,
    required this.day,
    required this.duration,
    required this.exercises,
    required this.type,
  });
}

class Exercise {
  final String id;
  final String name;
  final int sets;
  final int reps;
  final double weight; // in kg
  final String? notes;
  final bool isCompleted;
  final ExerciseType type;

  Exercise({
    required this.id,
    required this.name,
    required this.sets,
    required this.reps,
    required this.weight,
    this.notes,
    this.isCompleted = false,
    this.type = ExerciseType.weight,
  });

  Exercise copyWith({
    String? id,
    String? name,
    int? sets,
    int? reps,
    double? weight,
    String? notes,
    bool? isCompleted,
    ExerciseType? type,
  }) {
    return Exercise(
      id: id ?? this.id,
      name: name ?? this.name,
      sets: sets ?? this.sets,
      reps: reps ?? this.reps,
      weight: weight ?? this.weight,
      notes: notes ?? this.notes,
      isCompleted: isCompleted ?? this.isCompleted,
      type: type ?? this.type,
    );
  }
}

enum ExerciseType {
  weight,
  cardio,
  bodyweight,
  time,
}

class MuscleGroup {
```

```
  final String name;
  final String icon;
  final String status; // "Growing", "Maintaining", etc.
  final int currentReps;
  final int targetIncrease;
  final int totalReps;
  final double progress; // 0.0 to 1.0

  MuscleGroup({
    required this.name,
    required this.icon,
    required this.status,
    required this.currentReps,
    required this.targetIncrease,
    required this.totalReps,
    required this.progress,
  });
}

class UserStats {
  final double bodyWeight; // in lbs
  final DateTime lastWeightUpdate;
  final double volumeLifted; // in lbs
  final String period; // "Last 7 days", etc.
  final Map<DateTime, bool> workoutCalendar; // dates with workouts

  UserStats({
    required this.bodyWeight,
    required this.lastWeightUpdate,
    required this.volumeLifted,
    required this.period,
    required this.workoutCalendar,
  });
}

class CompletedWorkout {
  final String id;
  final String workoutName;
  final String workoutType;
  final DateTime completedAt;
  final Duration duration;
  final double volumeLifted;
  final int exercisesCompleted;
  final int totalExercises;

  CompletedWorkout({
    required this.id,
    required this.workoutName,
    required this.workoutType,
    required this.completedAt,
    required this.duration,
    required this.volumeLifted,
    required this.exercisesCompleted,
    required this.totalExercises,
  });
}

class BodyWeightEntry {
  final DateTime date;
  final double weight; // in lbs

  BodyWeightEntry({required this.date, required this.weight});
}

class PersonalRecord {
  final String exerciseName;
  final double weight;
  final int reps;
  final DateTime achievedAt;

  PersonalRecord({
    required this.exerciseName,
```

```
    required this.weight,
    required this.reps,
    required this.achievedAt,
  });
}

class ChatMessage {
  final String id;
  final String content;
  final bool isUser;
  final DateTime timestamp;

  ChatMessage({
    required this.id,
    required this.content,
    required this.isUser,
    required this.timestamp,
  });
}

class UserProfile {
  final String name;
  final String bio;
  final String goal; // e.g. "Muscle Gain & Strength"
  final String goalEmoji;
  final double targetWeight; // in lbs
  final int age;
  final double height; // in cm

  UserProfile({
    required this.name,
    required this.bio,
    required this.goal,
    required this.goalEmoji,
    required this.targetWeight,
    required this.age,
    required this.height,
  });

  UserProfile copyWith({
    String? name,
    String? bio,
    String? goal,
    String? goalEmoji,
    double? targetWeight,
    int? age,
    double? height,
  }) {
    return UserProfile(
      name: name ?? this.name,
      bio: bio ?? this.bio,
      goal: goal ?? this.goal,
      goalEmoji: goalEmoji ?? this.goalEmoji,
      targetWeight: targetWeight ?? this.targetWeight,
      age: age ?? this.age,
      height: height ?? this.height,
    );
  }
}
```

A.4 State management (lib/providers/workout_provider.dart)

```
import 'package:flutter/material.dart';
import '../models/workout.dart';

class WorkoutProvider with ChangeNotifier {
  List<Workout> _workouts = [];
  List<MuscleGroup> _muscleGroups = [];
  UserStats? _userStats;
  Workout? _currentWorkout;
  bool _isWorkoutInProgress = false;
  Duration _workoutDuration = Duration.zero;

  List<CompletedWorkout> _completedWorkouts = [];
  List<BodyWeightEntry> _bodyWeightHistory = [];
  List<PersonalRecord> _personalRecords = [];
  List<ChatMessage> _chatMessages = [];
  UserProfile _userProfile = UserProfile(
    name: 'Farish Sharif',
    bio: 'Fitness Enthusiast • Push/Pull/Cardio',
    goal: 'Muscle Gain & Strength',
    goalEmoji: '  ',
    targetWeight: 200.0,
    age: 24,
    height: 178.0,
  );

  int _currentExerciseIndex = 0;

  List<Workout> get workouts => _workouts;
  List<MuscleGroup> get muscleGroups => _muscleGroups;
  UserStats? get userStats => _userStats;
  Workout? get currentWorkout => _currentWorkout;
  bool get isWorkoutInProgress => _isWorkoutInProgress;
  Duration get workoutDuration => _workoutDuration;
  List<CompletedWorkout> get completedWorkouts => _completedWorkouts;
  List<BodyWeightEntry> get bodyWeightHistory => _bodyWeightHistory;
  List<PersonalRecord> get personalRecords => _personalRecords;
  List<ChatMessage> get chatMessages => _chatMessages;
  int get currentExerciseIndex => _currentExerciseIndex;
  UserProfile get userProfile => _userProfile;

  int get currentStreak {
    if (_userStats == null) return 0;
    final calendar = _userStats!.workoutCalendar;
    int streak = 0;
    final now = DateTime.now();
    for (int i = 0; i < 30; i++) {
      final date = DateTime(now.year, now.month, now.day - i);
      final hasWorkout = calendar.entries
          .where((e) =>
              e.key.year == date.year &&
              e.key.month == date.month &&
              e.key.day == date.day)
          .any((e) => e.value);
      if (hasWorkout) {
        streak++;
      } else if (i > 0) {
        break;
      }
    }
    return streak;
  }

  WorkoutProvider() {
    _initializeData();
  }

  void _initializeData() {
    _workouts = [
      Workout(
        id: '1',
```

```
      name: 'Chest + Tricep',
      day: 'Fridays',
      duration: 60,
      type: 'Push day',
      exercises: [
        Exercise(id: '1', name: 'Barbell Bench Press', sets: 5, reps: 8, weight: 80.0),
        Exercise(id: '2', name: 'Barbell Military Press', sets: 3, reps: 10, weight: 50.0),
        Exercise(id: '3', name: 'Dumbbell Incline Press', sets: 3, reps: 12, weight: 30.0),
        Exercise(id: '4', name: 'Dumbbell Lateral Raises', sets: 3, reps: 15, weight: 12.5),
        Exercise(id: '5', name: 'Dumbbell Tricep Extensions', sets: 3, reps: 12, weight: 20.0),
      ],
    ),
    Workout(
      id: '2',
      name: 'Back + Bicep + Legs',
      day: 'Mondays',
      duration: 75,
      type: 'Pull day',
      exercises: [
        Exercise(id: '6', name: 'Deadlifts', sets: 5, reps: 5, weight: 120.0),
        Exercise(id: '7', name: 'Pull-ups', sets: 4, reps: 8, weight: 0.0, type: ExerciseType.bodyweight),
        Exercise(id: '8', name: 'Barbell Rows', sets: 4, reps: 10, weight: 70.0),
        Exercise(id: '9', name: 'Barbell Curls', sets: 3, reps: 12, weight: 40.0),
        Exercise(id: '10', name: 'Leg Press', sets: 4, reps: 12, weight: 140.0),
      ],
    ),
    Workout(
      id: '3',
      name: 'Cardio & Core',
      day: 'Wednesdays',
      duration: 45,
      type: 'Cardio',
      exercises: [
        Exercise(id: '11', name: 'Treadmill Run', sets: 1, reps: 1, weight: 0.0, type: ExerciseType.cardio, notes: '5 km • 5:40 min/km'),
        Exercise(id: '12', name: 'Pull-ups (Chins)', sets: 4, reps: 8, weight: 0.0, type: ExerciseType.bodyweight),
        Exercise(id: '13', name: 'Planks', sets: 3, reps: 1, weight: 0.0, type: ExerciseType.time, notes: '3 x 30 sec'),
        Exercise(id: '14', name: 'Bicycle Crunches', sets: 3, reps: 20, weight: 0.0, type: ExerciseType.bodyweight),
        Exercise(id: '15', name: 'Stationary Bike', sets: 1, reps: 1, weight: 0.0, type: ExerciseType.cardio, notes: '10 min'),
      ],
    ),
  ];

  _muscleGroups = [
    MuscleGroup(name: 'Front Delts', icon: '  ', status: 'Growing', currentReps: 0, targetIncrease: 6, totalReps: 18, progress: 0.3),
    MuscleGroup(name: 'Chest', icon: '', status: 'Growing', currentReps: 0, targetIncrease: 6, totalReps: 18, progress: 0.4),
    MuscleGroup(name: 'Side Delts', icon: '  ', status: 'Growing', currentReps: 0, targetIncrease: 6, totalReps: 18, progress: 0.2),
    MuscleGroup(name: 'Back', icon: '  ', status: 'Maintaining', currentReps: 0, targetIncrease: 4, totalReps: 16, progress: 0.6),
    MuscleGroup(name: 'Biceps', icon: '  ', status: 'Growing', currentReps: 0, targetIncrease: 5, totalReps: 15, progress: 0.5),
    MuscleGroup(name: 'Legs', icon: '  ', status: 'Maintaining', currentReps: 0, targetIncrease: 8, totalReps: 24, progress: 0.35),
  ];

  _userStats = UserStats(
    bodyWeight: 190.0,
    lastWeightUpdate: DateTime.now().subtract(const Duration(minutes: 31)),
    volumeLifted: 3200.0,
    period: 'Last 7 days',
    workoutCalendar: _generateWorkoutCalendar(),
  );

  _bodyWeightHistory = _generateBodyWeightHistory();
  _personalRecords = _generatePersonalRecords();
  _completedWorkouts = _generateCompletedWorkouts();
  _initializeChatMessages();

  notifyListeners();
}

List<BodyWeightEntry> _generateBodyWeightHistory() {
  final now = DateTime.now();
  final weights = [195.0, 194.0, 193.5, 193.0, 192.0, 191.5, 191.0, 191.0, 190.5, 190.0, 190.0, 189.5];
  return List.generate(12, (i) {
    return BodyWeightEntry(
      date: DateTime(now.year, now.month - (11 - i), now.day),
      weight: weights[i],
```

```
    );
  });
 }

 List<PersonalRecord> _generatePersonalRecords() {
  final now = DateTime.now();
  return [
   PersonalRecord(exerciseName: 'Barbell Bench Press', weight: 100.0, reps: 5, achievedAt: now.subtract(const Duration(days: 7))),
   PersonalRecord(exerciseName: 'Deadlifts', weight: 140.0, reps: 3, achievedAt: now.subtract(const Duration(days: 14))),
   PersonalRecord(exerciseName: 'Barbell Rows', weight: 80.0, reps: 8, achievedAt: now.subtract(const Duration(days: 21))),
   PersonalRecord(exerciseName: 'Pull-ups', weight: 0.0, reps: 12, achievedAt: now.subtract(const Duration(days: 3))),
   PersonalRecord(exerciseName: 'Barbell Military Press', weight: 60.0, reps: 6, achievedAt: now.subtract(const Duration(days: 10))),
  ];
 }

 List<CompletedWorkout> _generateCompletedWorkouts() {
  final now = DateTime.now();
  return [
   CompletedWorkout(id: 'c1', workoutName: 'Chest + Tricep', workoutType: 'Push day', completedAt: now.subtract(const Duration(days:
1)), duration: const Duration(minutes: 62), volumeLifted: 3150, exercisesCompleted: 5, totalExercises: 5),
   CompletedWorkout(id: 'c2', workoutName: 'Back + Bicep + Legs', workoutType: 'Pull day', completedAt: now.subtract(const
Duration(days: 3)), duration: const Duration(minutes: 78), volumeLifted: 4200, exercisesCompleted: 4, totalExercises: 5),
   CompletedWorkout(id: 'c3', workoutName: 'Cardio & Core', workoutType: 'Cardio', completedAt: now.subtract(const Duration(days:
5)), duration: const Duration(minutes: 47), volumeLifted: 0, exercisesCompleted: 5, totalExercises: 5),
   CompletedWorkout(id: 'c4', workoutName: 'Chest + Tricep', workoutType: 'Push day', completedAt: now.subtract(const Duration(days:
8)), duration: const Duration(minutes: 59), volumeLifted: 3050, exercisesCompleted: 5, totalExercises: 5),
   CompletedWorkout(id: 'c5', workoutName: 'Back + Bicep + Legs', workoutType: 'Pull day', completedAt: now.subtract(const
Duration(days: 10)), duration: const Duration(minutes: 72), volumeLifted: 4000, exercisesCompleted: 5, totalExercises: 5),
  ];
 }

 void _initializeChatMessages() {
  _chatMessages = [
   ChatMessage(
    id: '0',
    content: "Hey! I'm your AI fitness coach  I'm here to help you crush your goals. Ask me anything about workouts, nutrition, or
recovery!",
    isUser: false,
    timestamp: DateTime.now().subtract(const Duration(minutes: 5)),
   ),
  ];
 }

 Map<DateTime, bool> _generateWorkoutCalendar() {
  final calendar = <DateTime, bool>{};
  final now = DateTime.now();
  for (int month = 0; month < 3; month++) {
   final date = DateTime(now.year, now.month - month, 1);
   final daysInMonth = DateTime(date.year, date.month + 1, 0).day;
   for (int day = 1; day <= daysInMonth; day++) {
    final currentDate = DateTime(date.year, date.month, day);
    calendar[currentDate] =
      (day % 3 == 0 || day % 7 == 0) && currentDate.isBefore(now);
   }
  }
  return calendar;
 }

 void startWorkout(Workout workout) {
  _currentWorkout = Workout(
   id: workout.id,
   name: workout.name,
   day: workout.day,
   duration: workout.duration,
   exercises: List.from(workout.exercises),
   type: workout.type,
  );
  _isWorkoutInProgress = true;
  _workoutDuration = Duration.zero;
  _currentExerciseIndex = 0;
  notifyListeners();
 }
```

```
void finishWorkout() {
  if (_currentWorkout != null) {
    final completed = CompletedWorkout(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      workoutName: _currentWorkout!.name,
      workoutType: _currentWorkout!.type,
      completedAt: DateTime.now(),
      duration: _workoutDuration,
      volumeLifted: _calculateVolumeLifted(_currentWorkout!.exercises),
      exercisesCompleted: _currentWorkout!.exercises.where((e) => e.isCompleted).length,
      totalExercises: _currentWorkout!.exercises.length,
    );
    _completedWorkouts.insert(0, completed);

    final today = DateTime.now();
    final todayKey = DateTime(today.year, today.month, today.day);
    final updatedCalendar = Map<DateTime, bool>.from(_userStats!.workoutCalendar);
    updatedCalendar[todayKey] = true;
    _userStats = UserStats(
      bodyWeight: _userStats!.bodyWeight,
      lastWeightUpdate: _userStats!.lastWeightUpdate,
      volumeLifted: _userStats!.volumeLifted + completed.volumeLifted,
      period: _userStats!.period,
      workoutCalendar: updatedCalendar,
    );
  }
  _currentWorkout = null;
  _isWorkoutInProgress = false;
  _workoutDuration = Duration.zero;
  _currentExerciseIndex = 0;
  notifyListeners();
}

double _calculateVolumeLifted(List<Exercise> exercises) {
  double total = 0;
  for (final e in exercises) {
    if (e.type == ExerciseType.weight) {
      total += e.weight * e.sets * e.reps;
    }
  }
  return total;
}

void updateWorkoutDuration(Duration duration) {
  _workoutDuration = duration;
  notifyListeners();
}

void toggleExerciseCompletion(String exerciseId) {
  if (_currentWorkout != null) {
    final exercises = List<Exercise>.from(_currentWorkout!.exercises);
    final idx = exercises.indexWhere((e) => e.id == exerciseId);
    if (idx != -1) {
      exercises[idx] = exercises[idx].copyWith(isCompleted: !exercises[idx].isCompleted);
      _currentWorkout = Workout(
        id: _currentWorkout!.id,
        name: _currentWorkout!.name,
        day: _currentWorkout!.day,
        duration: _currentWorkout!.duration,
        exercises: exercises,
        type: _currentWorkout!.type,
      );
      final completedCount = exercises.where((e) => e.isCompleted).length;
      _currentExerciseIndex = completedCount < exercises.length ? completedCount : exercises.length - 1;
      notifyListeners();
    }
  }
}

void addWorkout(Workout workout) {
  _workouts.add(workout);
  notifyListeners();
}
```

```
  void deleteWorkout(String id) {
    _workouts.removeWhere((w) => w.id == id);
    notifyListeners();
  }

  void logBodyWeight(double weight) {
    _bodyWeightHistory.add(BodyWeightEntry(date: DateTime.now(), weight: weight));
    _userStats = UserStats(
      bodyWeight: weight,
      lastWeightUpdate: DateTime.now(),
      volumeLifted: _userStats?.volumeLifted ?? 0,
      period: _userStats?.period ?? 'Last 7 days',
      workoutCalendar: _userStats?.workoutCalendar ?? {},
    );
    notifyListeners();
  }

  void updateProfile(UserProfile updated) {
    _userProfile = updated;
    notifyListeners();
  }

  void addChatMessage(String content, bool isUser) {
    _chatMessages.add(ChatMessage(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      content: content,
      isUser: isUser,
      timestamp: DateTime.now(),
    ));
    notifyListeners();
  }

  String generateCoachResponse(String userMessage) {
    final msg = userMessage.toLowerCase();

    if (msg.contains('bench') || msg.contains('chest')) {
      return "Great question! For bench press, focus on:\n\n• Keep your shoulder blades retracted\n• Drive your feet into the ground\n• Control the descent — 2 seconds down\n• Aim for progressive overload each week 📈\n\nYou're currently doing 80kg for 5×8. Consider bumping to 82.5kg next session!";
    } else if (msg.contains('protein') || msg.contains('diet') || msg.contains('nutrition') || msg.contains('eat')) {
      return "Nutrition is 70% of your results! Here's my advice:\n\n• Aim for **1g protein per lb** of body weight (190g for you)\n• Prioritize whole foods: chicken, eggs, fish, legumes\n• Eat 2-3 hours before training\n• Stay hydrated — 2-3L of water daily 💧\n\nWant a sample meal plan?";
    } else if (msg.contains('rest') || msg.contains('recover') || msg.contains('sleep')) {
      return "Recovery is where the gains happen! 😴💤\n\n• Aim for **7-9 hours** of sleep per night\n• Rest 48-72 hours between training the same muscle group\n• Active recovery (walking, stretching) on rest days\n• Consider foam rolling post-workout\n\nYour schedule looks solid — 3 days on works well!";
    } else if (msg.contains('cardio') || msg.contains('run') || msg.contains('endurance')) {
      return "Cardio tips for you:\n\n• Keep cardio **separate** from heavy lifting sessions\n• 2-3x per week is plenty for fat loss while maintaining muscle\n• Zone 2 cardio (conversational pace) is most effective for fat oxidation\n• Your 5km runs are great — try to maintain that pace! 🏃\n\nConsider adding HIIT once a week for extra calorie burn.";
    } else if (msg.contains('motivation') || msg.contains('tired') || msg.contains('lazy')) {
      return "We all feel that way sometimes. Here's what works:\n\n• **Just start** — commit to 10 minutes, you'll usually finish\n• Track your progress (you're already doing this! 📊)\n• Remember your *why* — why did you start?\n• Your streak is looking great, don't break it! 🔥\n\nYou've got this. The hardest part is walking through the gym door 🚪";
    } else if (msg.contains('weight loss') || msg.contains('fat') || msg.contains('cut')) {
      return "For fat loss while preserving muscle:\n\n• Create a **300-500 cal deficit** — not too aggressive\n• Keep protein HIGH (190g+ per day at your weight)\n• Maintain your lifting intensity\n• Add 1-2 cardio sessions per week\n• Weigh yourself daily, track the **weekly average**\n\nYou're at 190 lbs — what's your target weight?";
    } else if (msg.contains('deadlift')) {
      return "Deadlifts are king! 👑 Some form cues:\n\n• Hip hinge, not a squat — push the floor away\n• Bar stays close to your body throughout\n• Lock out with glutes, not lower back\n• Brace your core before each rep\n\nYou're pulling 120kg for 5×5 — impressive! Ready to try 125kg?";
    } else if (msg.contains('hi') || msg.contains('hello') || msg.contains('hey')) {
      return "Hey there! 👋 Ready to crush your goals today?\n\nI can help you with:\n• **Workout advice** — form, programming, exercises\n• **Nutrition** — protein, meal timing, supplements\n• **Recovery** — sleep, rest days, injury prevention\n• **Motivation** — when you need a push\n\nWhat's on your mind?";
    } else if (msg.contains('supplement') || msg.contains('creatine') || msg.contains('protein powder')) {
      return "Supplement advice:\n\n• **Creatine** — 5g/day, most studied supplement for strength 💪\n• **Protein powder** — use to hit daily protein goals, not replace food\n• **Caffeine** — great pre-workout, but don't overdo it\n• **Vitamin D** — most people are deficient, huge health benefits\n\nThese 4 are evidence-backed. Everything else is mostly marketing 😅";
    } else {
```

```
      return "That's a great question! As your AI coach, I'd recommend:\n\n• Focus on **consistency** over perfection\n• Progressive overload is the key to gains\n• Track everything — what gets measured, gets managed\n• Listen to your body — rest when needed\n\nFeel free to ask me more specific questions about your workouts, nutrition, or recovery! ♂";
    }
  }
}
```

A.5 Main navigation (lib/screens/main_navigation_screen.dart)

```
import 'package:flutter/material.dart';
import 'workouts_screen.dart';
import 'calendar_screen.dart';
import 'stats_screen.dart';
import 'chat_screen.dart';
import 'profile_screen.dart';

class MainNavigationScreen extends StatefulWidget {
  const MainNavigationScreen({super.key});

  @override
  State<MainNavigationScreen> createState() => _MainNavigationScreenState();
}

class _MainNavigationScreenState extends State<MainNavigationScreen> {
  int _currentIndex = 0;

  final List<Widget> _screens = [
    const WorkoutsScreen(),
    const CalendarScreen(),
    const StatsScreen(),
    const ChatScreen(),
    const ProfileScreen(),
  ];

  final List<_NavItem> _navItems = [
    _NavItem(Icons.grid_view_rounded, Icons.grid_view_rounded, 'Workouts'),
    _NavItem(Icons.calendar_today_rounded, Icons.calendar_today_rounded, 'Calendar'),
    _NavItem(Icons.bar_chart_rounded, Icons.bar_chart_rounded, 'Stats'),
    _NavItem(Icons.chat_bubble_outline_rounded, Icons.chat_bubble_rounded, 'Coach'),
    _NavItem(Icons.person_outline_rounded, Icons.person_rounded, 'Profile'),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: IndexedStack(index: _currentIndex, children: _screens),
      bottomNavigationBar: Container(
        decoration: BoxDecoration(
          color: const Color(0xFF111111),
          border: Border(top: BorderSide(color: Colors.white.withOpacity(0.07))),
        ),
        child: SafeArea(
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 8),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.spaceAround,
              children: List.generate(_navItems.length, (index) {
                return _buildNavItem(index);
              }),
            ),
          ),
        ),
      ),
    );
  }

  Widget _buildNavItem(int index) {
    final isSelected = _currentIndex == index;
    final item = _navItems[index];

    return GestureDetector(
      onTap: () => setState(() => _currentIndex = index),
      behavior: HitTestBehavior.opaque,
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 200),
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
        decoration: BoxDecoration(
          color: isSelected ? const Color(0xFF2A2A2A) : Colors.transparent,
```

```
          borderRadius: BorderRadius.circular(12),
        ),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(
              isSelected ? item.activeIcon : item.icon,
              color: isSelected ? Colors.white : const Color(0xFF505 050),
              size: 22,
            ),
            const SizedBox(height: 3),
            Text(
              item.label,
              style: TextStyle(
                color: isSelected ? Colors.white : const Color(0xFF505 050),
                fontSize: 10,
                fontWeight: isSelected ? FontWeight.w600 : FontWeight.normal,
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class _NavItem {
  final IconData icon;
  final IconData activeIcon;
  final String label;

  const _NavItem(this.icon, this.activeIcon, this.label);
}
```

A.6 Workouts screen (lib/screens/workouts_screen.dart)

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/workout_provider.dart';
import '../models/workout.dart';
import '../widgets/workout_card.dart';
import '../widgets/stats_card.dart';
import '../widgets/calendar_card.dart';
import 'workout_detail_screen.dart';
import 'add_workout_screen.dart';


class WorkoutsScreen extends StatelessWidget {
  const WorkoutsScreen({super.key});


  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Theme.of(context).scaffoldBackgroundColor,
      body: SafeArea(
        child: Consumer<WorkoutProvider>(
          builder: (context, workoutProvider, child) {
            return CustomScrollView(
              slivers: [
                // Header
                SliverToBoxAdapter(
                  child: Padding(
                    padding: const EdgeInsets.all(20.0),
                    child: Row(
                      mainAxisAlignment: MainAxisAlignment.spaceBetween,
                      children: [
                        const Text(
                          'Workouts',
                          style: TextStyle(
                            fontSize: 32,
                            fontWeight: FontWeight.bold,
                            color: Colors.white,
                          ),
                        ),
                        Row(
                          children: [
                            // Filter button (placeholder)
                            Container(
                              width: 40,
                              height: 40,
                              decoration: const BoxDecoration(
                                color: Color(0xFF2A2A2A),
                                shape: BoxShape.circle,
                              ),
                              child: const Icon(Icons.filter_list, color: Colors.white, size: 20),
                            ),
                            const SizedBox(width: 10),
                            // Add workout button
                            GestureDetector(
                              onTap: () {
                                Navigator.push(
                                  context,
                                  MaterialPageRoute(builder: (_) => const AddWorkoutScreen()),
                                );
                              },
                              child: Container(
                                width: 40,
                                height: 40,
                                decoration: const BoxDecoration(
                                  color: Colors.white,
                                  shape: BoxShape.circle,
                                ),
                                child: const Icon(Icons.add, color: Colors.black, size: 22),
                              ),
                            ),
                          ],
                        ),
                      ],
                    ),
                  ),
```

```
  ),
),

// Stats row (body weight + calendar)
SliverPadding(
  padding: const EdgeInsets.symmetric(horizontal: 20.0),
  sliver: SliverToBoxAdapter(
    child: Row(
      children: [
        Expanded(
          child: workoutProvider.workouts.isNotEmpty
              ? WorkoutCard(
                  workout: workoutProvider.workouts[0],
                  onTap: () => _navigateToWorkoutDetail(context, workoutProvider.workouts[0]),
                )
              : const SizedBox.shrink(),
        ),
        const SizedBox(width: 12),
        Expanded(
          child: StatsCard(
            title: 'Body weight',
            value: '${workoutProvider.userStats?.bodyWeight.toInt() ?? 0} lbs',
            subtitle: _formatTimeAgo(workoutProvider.userStats?.lastWeightUpdate ?? DateTime.now()),
          ),
        ),
      ],
    ),
  ),
),

const SliverToBoxAdapter(child: SizedBox(height: 12)),

// Calendar card
SliverPadding(
  padding: const EdgeInsets.symmetric(horizontal: 20.0),
  sliver: SliverToBoxAdapter(
    child: CalendarCard(
      workoutCalendar: workoutProvider.userStats?.workoutCalendar ?? {},
    ),
  ),
),

const SliverToBoxAdapter(child: SizedBox(height: 12)),

// All workouts list
SliverPadding(
  padding: const EdgeInsets.symmetric(horizontal: 20.0),
  sliver: SliverList(
    delegate: SliverChildBuilderDelegate(
      (context, index) {
        if (index == 0) {
          // Volume card
          return Column(
            children: [
              StatsCard(
                title: 'Volume lifted',
                value: '${workoutProvider.userStats?.volumeLifted.toInt() ?? 0} lbs',
                subtitle: workoutProvider.userStats?.period ?? '',
              ),
              const SizedBox(height: 12),
              if (workoutProvider.workouts.length > 1)
                WorkoutCard(
                  workout: workoutProvider.workouts[1],
                  onTap: () => _navigateToWorkoutDetail(context, workoutProvider.workouts[1]),
                ),
              const SizedBox(height: 12),
            ],
          );
        }
        // Remaining workouts from index 2 onward
        final wIndex = index + 1;
        if (wIndex >= workoutProvider.workouts.length) return const SizedBox.shrink();
        return Padding(
          padding: const EdgeInsets.only(bottom: 12),
```

```
                    child: WorkoutCard(
                      workout: workoutProvider.workouts[wIndex],
                      onTap: () => _navigateToWorkoutDetail(context, workoutProvider.workouts[wIndex]),
                    ),
                  );
                },
                childCount: (workoutProvider.workouts.length - 1).clamp(1, 100),
              ),
            ),
          ),

          const SliverToBoxAdapter(child: SizedBox(height: 100)),
        ],
      );
    },
  ),
 ),
 );
}

 void _navigateToWorkoutDetail(BuildContext context, Workout workout) {
  Navigator.push(
   context,
   MaterialPageRoute(builder: (context) => WorkoutDetailScreen(workout: workout)),
  );
 }

 String _formatTimeAgo(DateTime dateTime) {
  final now = DateTime.now();
  final difference = now.difference(dateTime);
  if (difference.inMinutes < 60) return '${difference.inMinutes} min ago';
  if (difference.inHours < 24) return '${difference.inHours}h ago';
  return '${difference.inDays}d ago';
 }
}
```

A.7 Workout card widget (lib/widgets/workout_card.dart)

```
import 'package:flutter/material.dart';
import '../models/workout.dart';

class WorkoutCard extends StatelessWidget {
  final Workout workout;
  final VoidCallback onTap;

  const WorkoutCard({
    super.key,
    required this.workout,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: Container(
        padding: const EdgeInsets.all(16),
        decoration: BoxDecoration(
          color: const Color(0xFF1A1A1A),
          borderRadius: BorderRadius.circular(16),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Container(
                  width: 32,
                  height: 32,
                  decoration: BoxDecoration(
                    color: Colors.white,
                    shape: BoxShape.circle,
                    border: Border.all(color: const Color(0xFF2A2A2A), width: 2),
                  ),
                  child: Center(
                    child: Text(
                      workout.id,
                      style: const TextStyle(
                        color: Colors.black,
                        fontWeight: FontWeight.bold,
                        fontSize: 14,
                      ),
                    ),
                  ),
                ),
                const Spacer(),
                const Icon(
                  Icons.more_horiz,
                  color: Colors.white,
                  size: 20,
                ),
              ],
            ),
            const SizedBox(height: 12),
            Text(
              workout.name,
              style: const TextStyle(
                color: Colors.white,
                fontSize: 16,
                fontWeight: FontWeight.w600,
              ),
            ),
            const SizedBox(height: 4),
            Text(
              workout.day,
              style: const TextStyle(
                color: Color(0xFFB0B0B0),
                fontSize: 14,
              ),
            ),
```

```
          ],
        ),
      ),
    );
  }
}
```

A.8 Muscle group card widget (lib/widgets/muscle_group_card.dart)

```
import 'package:flutter/material.dart';
import '../models/workout.dart';


class MuscleGroupCard extends StatelessWidget {
  final MuscleGroup muscleGroup;


  const MuscleGroupCard({super.key, required this.muscleGroup});


  @override
  Widget build(BuildContext context) {
    return Container(
      width: 120,
      height: 120,
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: const Color(0xFF1A1A1A),
        borderRadius: BorderRadius.circular(16),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        mainAxisSize: MainAxisSize.min,
        children: [
          // Icon
          Container(
            width: 28,
            height: 28,
            decoration: const BoxDecoration(
              color: Colors.white,
              shape: BoxShape.circle,
            ),
            child: const Center(
              child: Text('  ', style: TextStyle(fontSize: 14)),
            ),
          ),


          const SizedBox(height: 8),


          // Name
          Text(
            muscleGroup.name,
            style: const TextStyle(
              color: Colors.white,
              fontSize: 12,
              fontWeight: FontWeight.w600,
            ),
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
          ),


          const SizedBox(height: 4),


          // Status
          Row(
            children: [
              const Icon(Icons.trending_up, color: Color(0xFF00FF00), size: 10),
              const SizedBox(width: 2),
              const Text(
                'Growing',
                style: TextStyle(color: Color(0xFF00FF00), fontSize: 10),
              ),
            ],
          ),


          const SizedBox(height: 6),


          // Progress dots
          Row(
            children: List.generate(5, (index) {
              return Container(
```

```
                width: 3,
                height: 3,
                margin: const EdgeInsets.only(right: 3),
                decoration: BoxDecoration(
                  color: index < (muscleGroup.progress * 5).round()
                      ? Colors.white
                      : const Color(0xFF2A2A2A),
                  shape: BoxShape.circle,
                ),
              );
            }),
          ),

          const Spacer(),

          // Stats
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Text(
                '${muscleGroup.currentReps}',
                style: const TextStyle(color: Colors.white, fontSize: 10),
              ),
              Text(
                '+${muscleGroup.targetIncrease}',
                style: const TextStyle(color: Color(0xFF00FF00), fontSize: 10),
              ),
              Text(
                '${muscleGroup.totalReps}',
                style: const TextStyle(color: Colors.white, fontSize: 10),
              ),
            ],
          ),
        ],
      ),
    );
  }
}
```